课后作业

代码+报告打包提交

文件名 HW-2-姓名1-姓名2-姓名3.zip

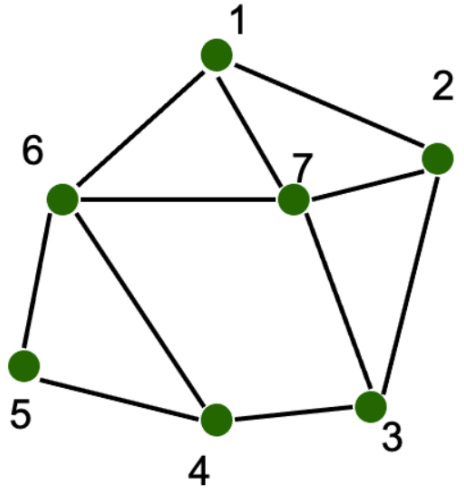
05-09提交给学委

作业编号 2
满分5+10分

问题1.使用回溯算法求解着色问题(5分)

给定右图.

1. 阅读定理4.4，使用“G的△(G)+1正常点着色算法”，手动计算一种可行的着色方案(无需优化最小着色数). (2分)
2. 阅读书本P81 例4.16，使用整数规划模型求解最小着色数. (3分)



问题2.DeepSeek-Coder代码数据处理(10分)

1. 阅读 DeepSeek-Coder 论文
2. 研究 Algorithm 1 & Sec. 2.2, 项目代码文件依赖图的序列化过程
   - 例如, 如果A.py 中 import B.py, 也就是说A依赖于B, 则图中有 $A \leftarrow B$ 的有向边. 由2.2节所述, 那么B文件应当排列在A之前.
3. 复现论文中所述 Topological-Sort 方法, 给出一个最优的文件依赖序列, 使得任一文件的依赖文件集合, 尽量排列在该文件之前
   - 例如, Main.py import Person.py, Person.py import Company.py, 那么这是一个链式图, 应当满足 $C \rightarrow P \rightarrow M$, 则返回[C, P, M]作为一个最优序列
4. 下载开源项目代码库，查看MBPP文件夹中所有.py文件（包括子文件夹），构建文件依赖关系的最优拓扑序列，画出拓扑图。
5. 验证满足：具有明确依赖关系的文件，基本满足序列中的前后关系. 如果有不满足的文件pair, 请列出.

2.2. Dependency Parsing

In previous works (Chen et al., 2021; Li et al., 2023; Nijkamp et al., 2022; Roziere et al., 2023), large language models for code are mainly pre-trained on file-level source code, which ignores the dependencies between different files in a project. However, in practical applications, such models struggle to effectively scale to handle entire project-level code scenarios. Therefore, we

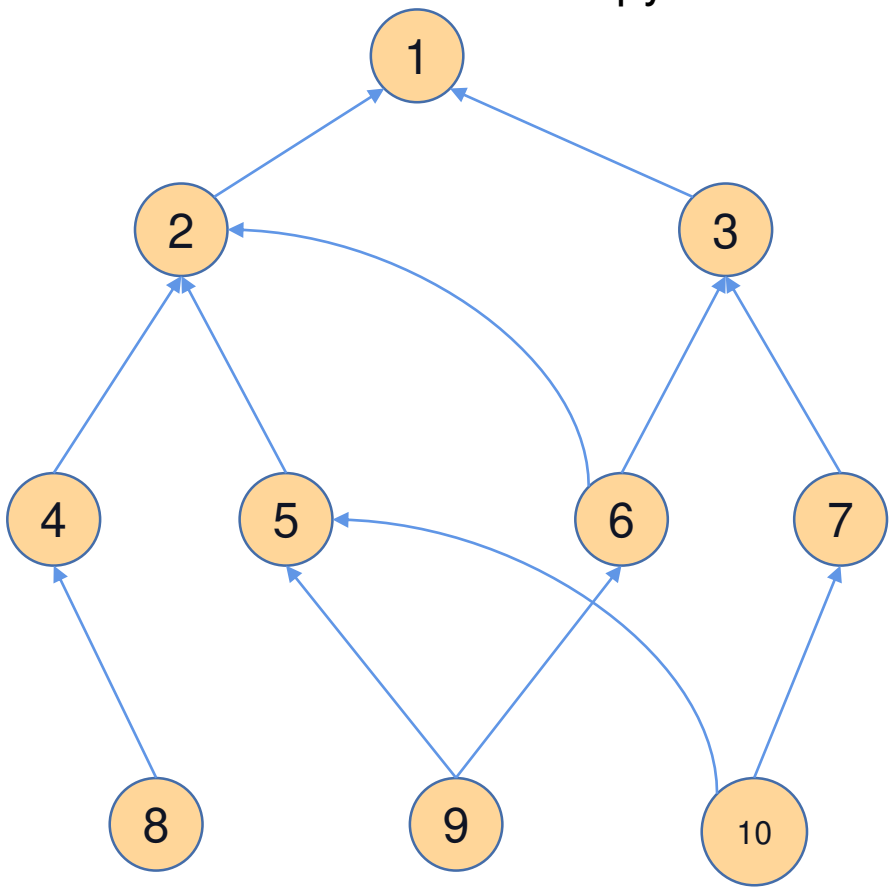
```
allResults ← []
for each subgraph in subgraphs do
    results ← []
    while length(results) ≠ NumberOfNodes(subgraph) do
        file ← argmin({inDegree[file] | file ∈ subgraph and file ∉ results})
        for each node in graphs[file] do
            inDegree[node] ← inDegree[node] − 1
        end for
        results.append(file)
    end while
    allResults.append(results)
end for

return allResults
end procedure
```

will consider how to leverage the dependencies between files within the same repository in this step. Specifically, we first parse the dependencies between files and then arrange these files in an order that ensures the context each file relies on is placed before that file in the input sequence. By aligning the files in accordance with their dependencies, our dataset more accurately represents real coding practices and structures. This enhanced alignment not only makes our dataset more relevant but also potentially increases the practicality and applicability of the model in handling project-level code scenarios. It's worth noting that we only consider the invocation relationships between files and use regular expressions to extract them, such as **"import"** in Python, **"using"** in C#, and **"include"** in C.

The algorithm 1 describes a topological sort for dependency analysis on a list of files within the same project. Initially, it sets up two data structures: an empty adjacency list named **"graphs"** to represent dependencies between files and an empty dictionary called **"inDegree"** for storing the in-degrees of each file. The algorithm then iterates over each file pair to identify depen-

main.py



代码文件拓扑依赖示意图。请根据MBPP文件夹，打印实际节点拓扑图及对应的文件名称。

作业纪律:

- 可参考网络代码, 请规范的进行引用.
- 代码请进行注释.
- **任何学术不端行为, 得0分.**
- **任何AI代码部分，注明prompt、工具（如模型名称，agent平台），用到任何Skill请注明.**